

ARTIFICIAL INTELLIGENCE

HSU

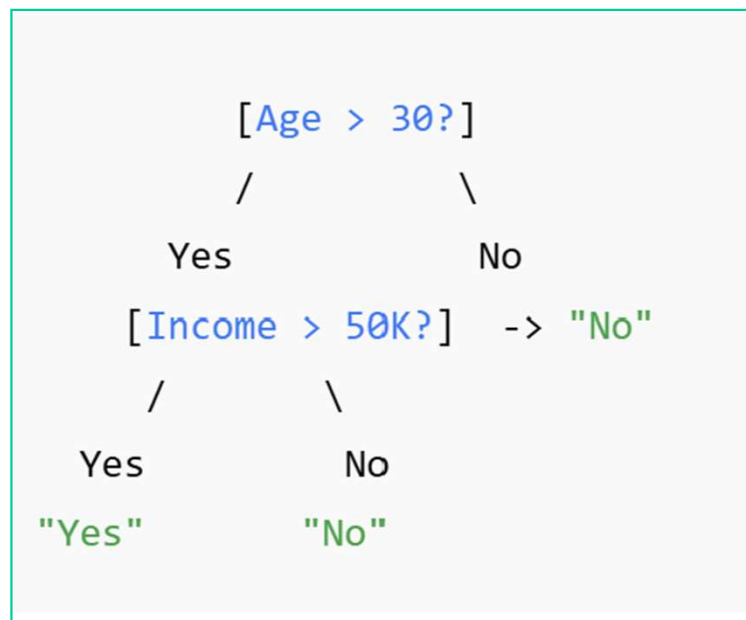
TS. Nguyen Quoc Minh

Decision Trees

Definition

- A **decision tree (DT)** is a type of **machine learning model** that represents decisions and their possible consequences in the form of a **tree-like structure**.

DT of customers buying a product:

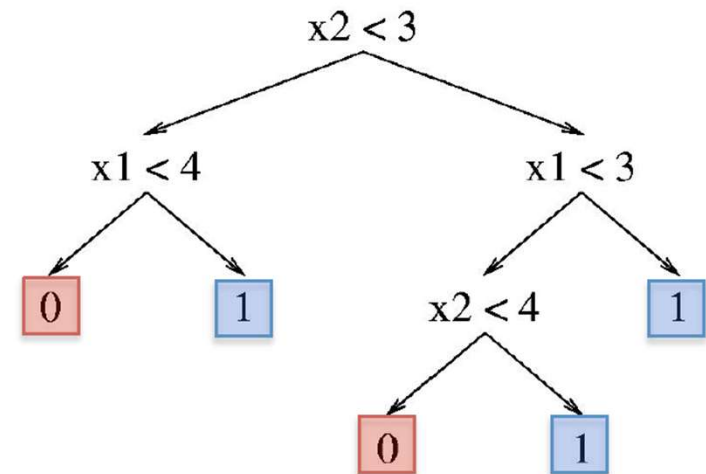
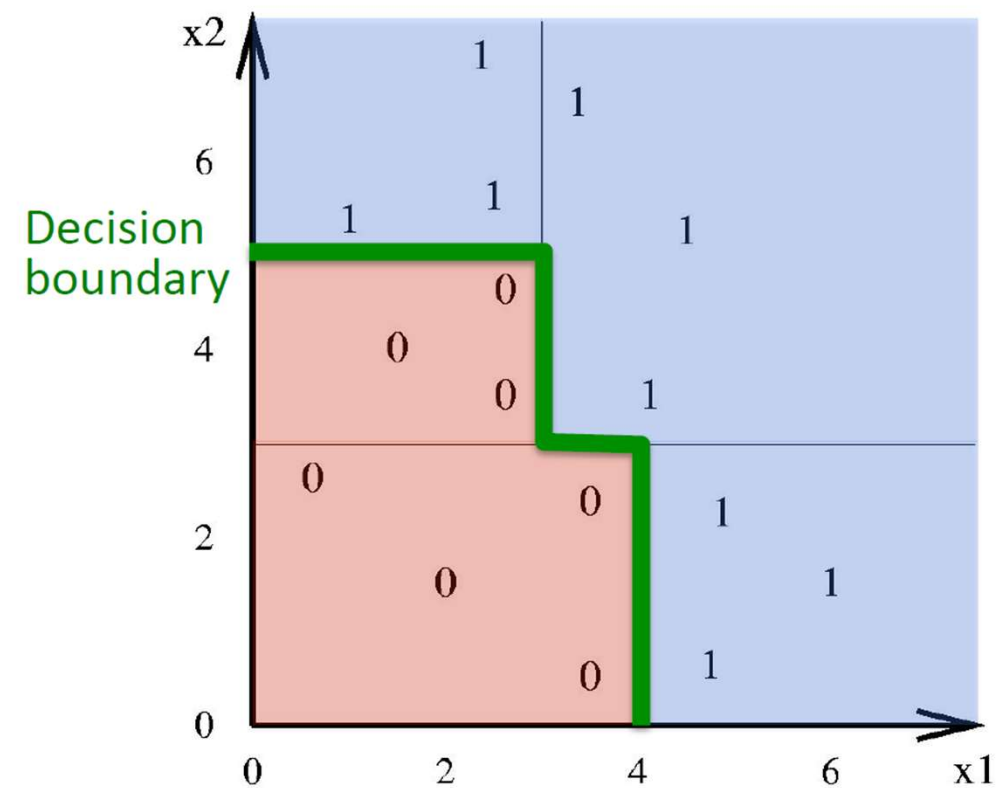


So if a person is over 30 and earns more than 50K, then?

→ prediction is "Yes."

Decision Tree Decision Boundaries

Decision trees divide the feature space into axis-parallel rectangles, and label each rectangle with one of the K classes.



1

How to build such a DT?

Training Data Example: Goal is to Predict When This Player Will Play Tennis?

PlayTennis: training examples

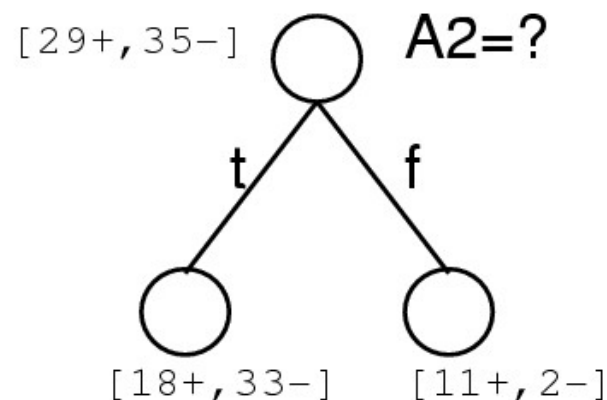
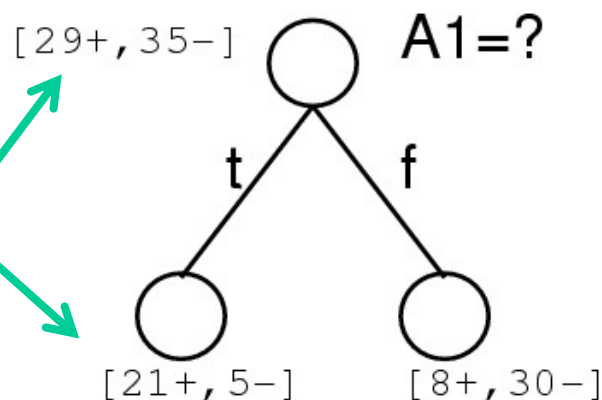
Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Choosing the **Best** Attribute

A1 and A2 are “attributes” (i.e. features or inputs).

Which attribute is best?

Number +
and – examples
before and after
a split.



- We will look at **Information Gain**.

Entropy

- Entropy measures the randomness/uncertainty in the data
- Let's consider a set S of examples with C many classes. Entropy of this set:

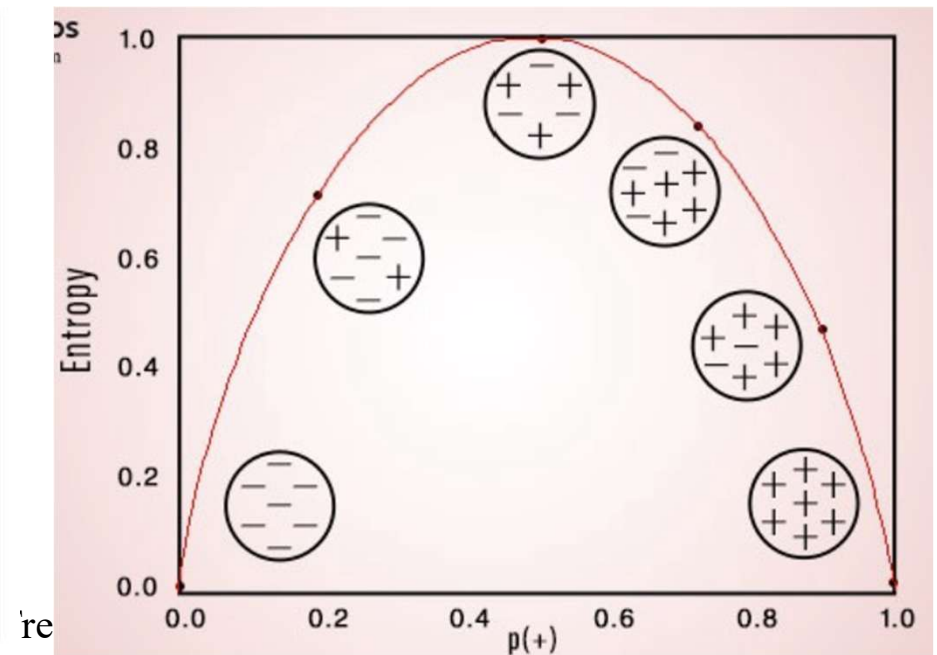
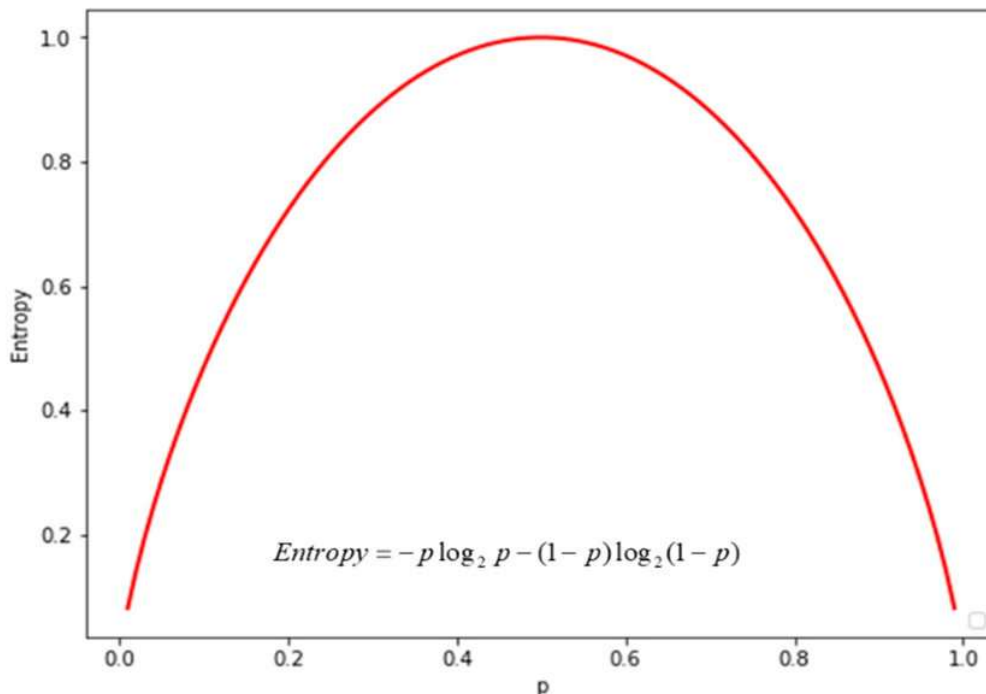
$$H(S) = - \sum_{c \in C} p_c \log_2 p_c$$

- p_c is the probability that an element of S belongs to class c
 - .. basically, the fraction of elements of S belonging to class c
- Intuition: Entropy is a measure of the “degree of surprise”
 - Some dominant classes $\implies$ small entropy (less uncertainty)
 - Equiprobable classes $\implies$ high entropy (more uncertainty)
- Entropy denotes the average number of bits needed to encode S

Entropy

- $p_{\oplus}$ is the proportion of positive examples in S
- $p_{\ominus}$ is the proportion of negative examples in S
- Entropy measures the impurity of S

$$\text{Entropy}(S) \equiv -p_{\oplus} \log_2 p_{\oplus} - p_{\ominus} \log_2 p_{\ominus}$$

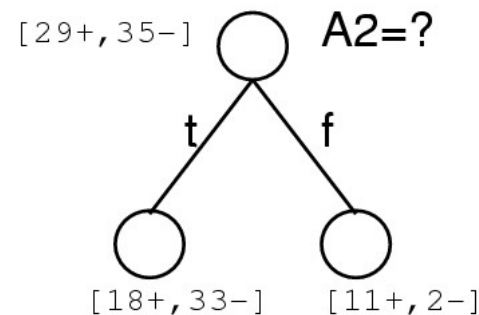
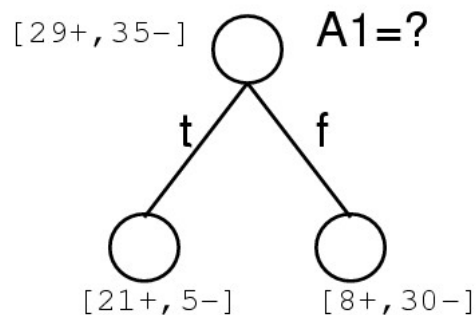


- Binary classification (2 classes):
Entropy is only restricted to a value between 0 and 1.
- Multi-class Systems: The maximum entropy is $\log_2(n)$
- Discrete Shannon entropy is always non-negative because Probability is in $[0, 1]$, making $-\log p \geq 0$

Information Gain

$Gain(S, A)$ = expected reduction in entropy due to sorting on A

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$



Information Gain

- Let's assume each element of S consists of a set of features
- Information Gain (IG) on a feature F

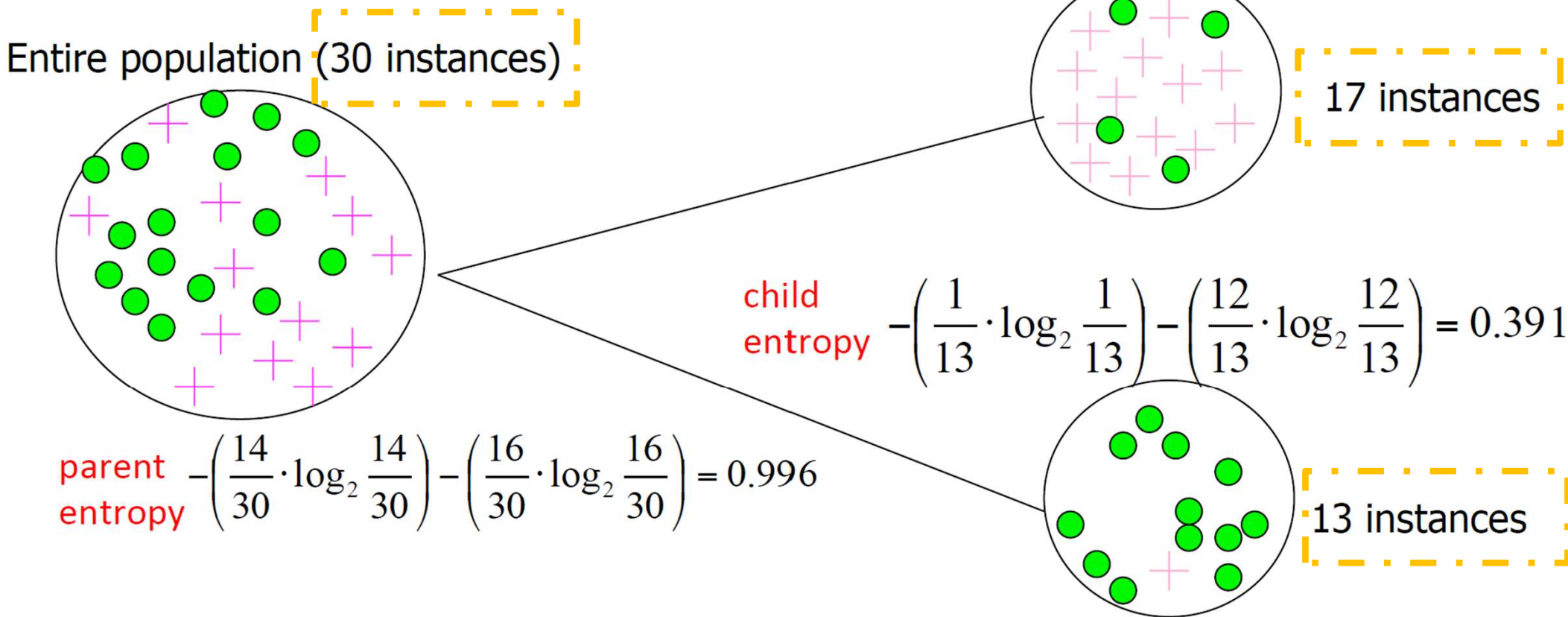
$$IG(S, F) = H(S) - \sum_{f \in F} \frac{|S_f|}{|S|} H(S_f)$$

- S_f number of elements of S with feature F having *value* f
- $IG(S, F)$ measures the **increase in our certainty** about S once we know the value of F
- $IG(S, F)$ denotes the number of bits saved while encoding S once we know the value of the feature F

Calculating Information Gain

Information Gain = entropy(parent) – [average entropy(children)]

child entropy $-\left(\frac{13}{17} \cdot \log_2 \frac{13}{17}\right) - \left(\frac{4}{17} \cdot \log_2 \frac{4}{17}\right) = 0.787$



(Weighted) Average Entropy of Children = $\left(\frac{17}{30} \cdot 0.787\right) + \left(\frac{13}{30} \cdot 0.391\right) = 0.615$

Information Gain = 0.996 - 0.615 = 0.38

PlayTennis: training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

What is the entropy of the examples before we select a feature for the root node of the tree?

There are 14 examples: 9 positive, 5 negative. Applying the formula gives

$$\begin{aligned} H_{\text{before}} &= - \left(\frac{9}{14} \log_2 \left(\frac{9}{14} \right) + \frac{5}{14} \log_2 \left(\frac{5}{14} \right) \right) \\ &= - \left(\frac{9}{14} (-0.637) + \frac{5}{14} (-1.485) \right) \\ &= -(-0.939) \\ &\approx 0.94. \end{aligned}$$

What is the expected information gain if we select **Outlook** as the root node of the tree?

Solution: Testing Outlook yields three branches:

$$\text{Outlook} = \begin{cases} \text{Sunny} & 2+ & 3- & 5 \text{ total} \\ \text{Overcast} & 4+ & 0- & 4 \text{ total} \\ \text{Rain} & 3+ & 2- & 5 \text{ total} \end{cases}$$

The expected information gain is

$$\begin{aligned} \text{Gain}(\text{Outlook}) &= 0.94 - \left(\frac{5}{14} \cdot I\left(\frac{2}{5}, \frac{3}{5}\right) + \frac{4}{14} \cdot I\left(\frac{4}{4}, \frac{0}{4}\right) + \frac{5}{14} \cdot I\left(\frac{3}{5}, \frac{2}{5}\right) \right) \\ &= 0.94 - \left(\frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) \right) \\ &= 0.94 - 0.694 \\ &= 0.247. \end{aligned}$$

What is the expected information gain
if we select **Humidity** as the root
node of the tree?

Solution: Testing Humidity yields two branches:

$$\text{Humidity} = \begin{cases} \text{Normal} & 6+ & 1- & 7 \text{ total} \\ \text{High} & 3+ & 4- & 7 \text{ total} \end{cases}$$

The expected information gain is

$$\begin{aligned} \text{Gain}(\text{Humidity}) &= 0.94 - \left(\frac{7}{14} \cdot I\left(\frac{6}{7}, \frac{1}{7}\right) + \frac{7}{14} \cdot I\left(\frac{3}{7}, \frac{4}{7}\right) \right) \\ &= 0.94 - \left(\frac{7}{14}(0.592) + \frac{7}{14}(0.985) \right) \\ &= 0.94 - 0.789 \\ &= 0.151. \end{aligned}$$

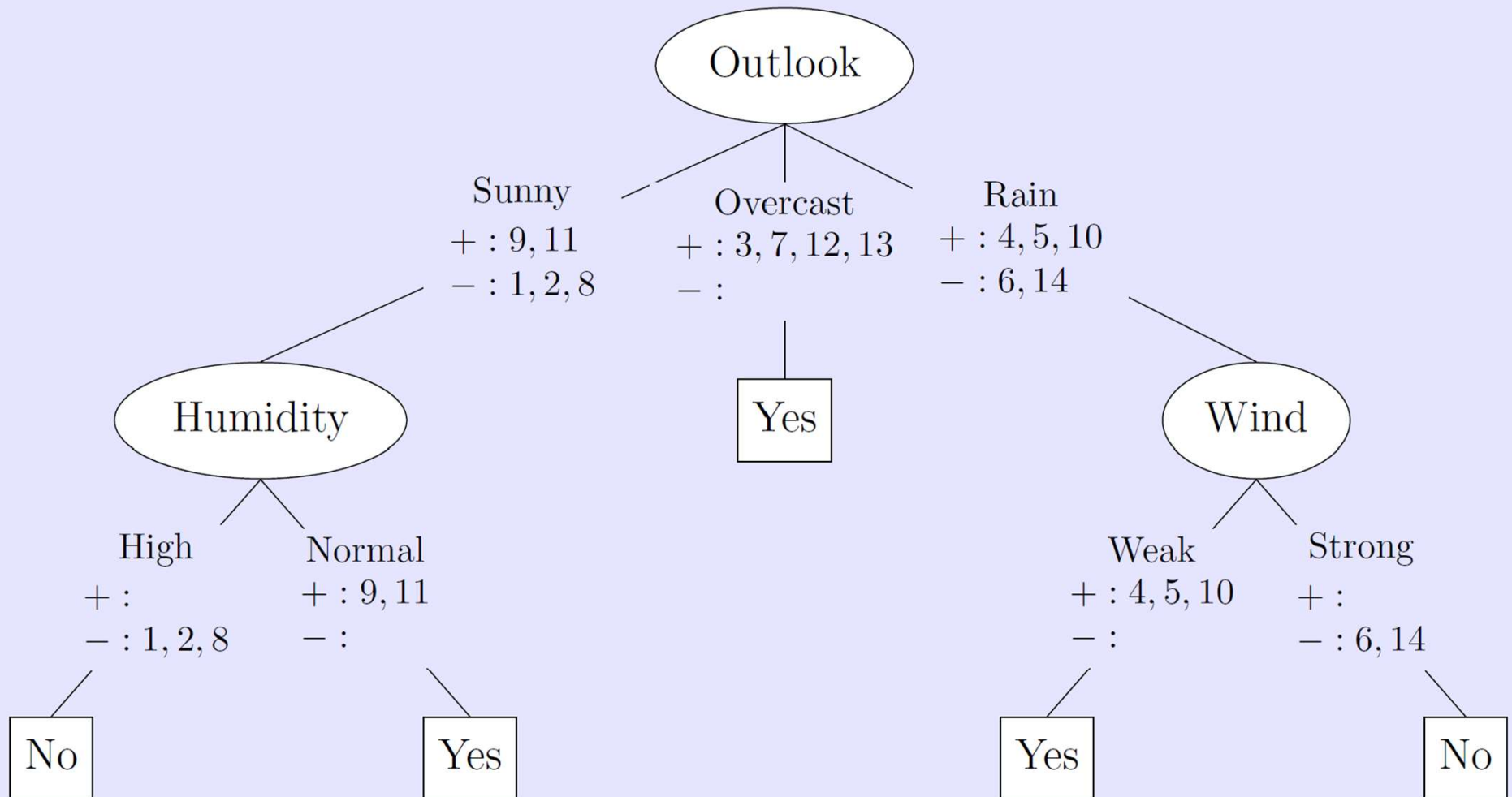
For the root node, we can choose from all four features to test:

$$\text{Gain}(\mathbf{Outlook}) = \mathbf{0.247} \quad \text{Gain}(\text{Humidity}) = 0.151$$

$$\text{Gain}(\text{Temp}) = 0.029 \quad \text{Gain}(\text{Wind}) = 0.048$$

We pick Outlook since it has the greatest expected information gain.

Final DT



Recall that we wanted a shallow, small tree, and that's exactly what we got.

When to stop SPLITTING?

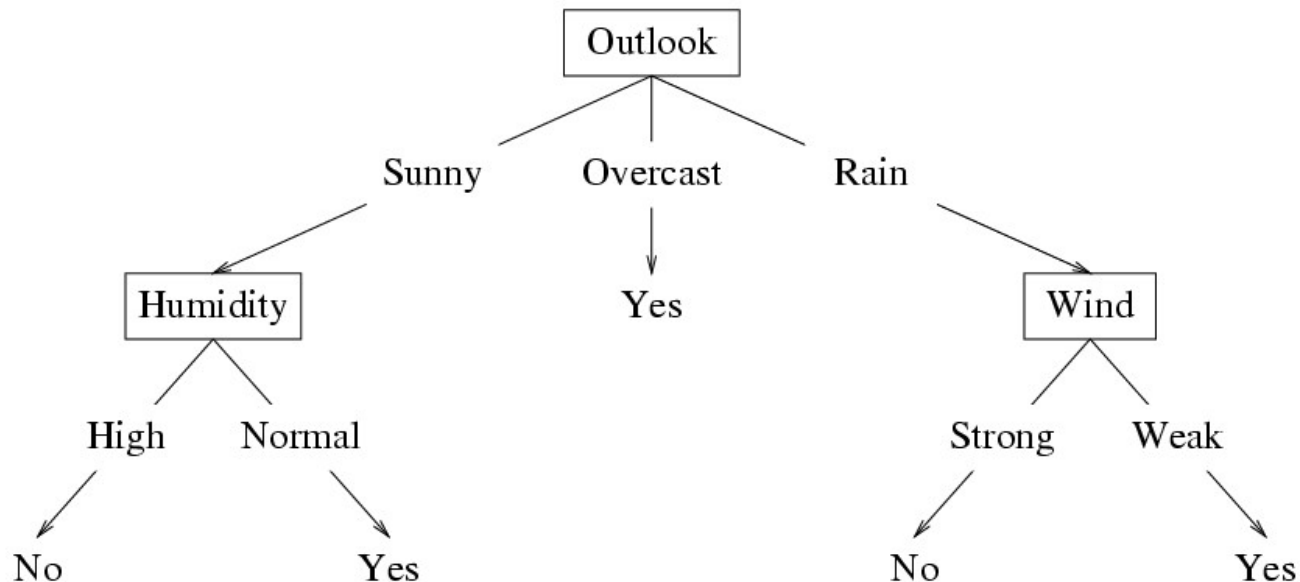
Decision trees should stop splitting when

- nodes become **pure** (homogeneous),
- a **maximum depth** is reached,
- the number of samples in a node is too small,
- or further splitting does not significantly reduce impurity (early stopping/pre-pruning)

These conditions prevent overfitting and ensure the model generalizes well to new data.

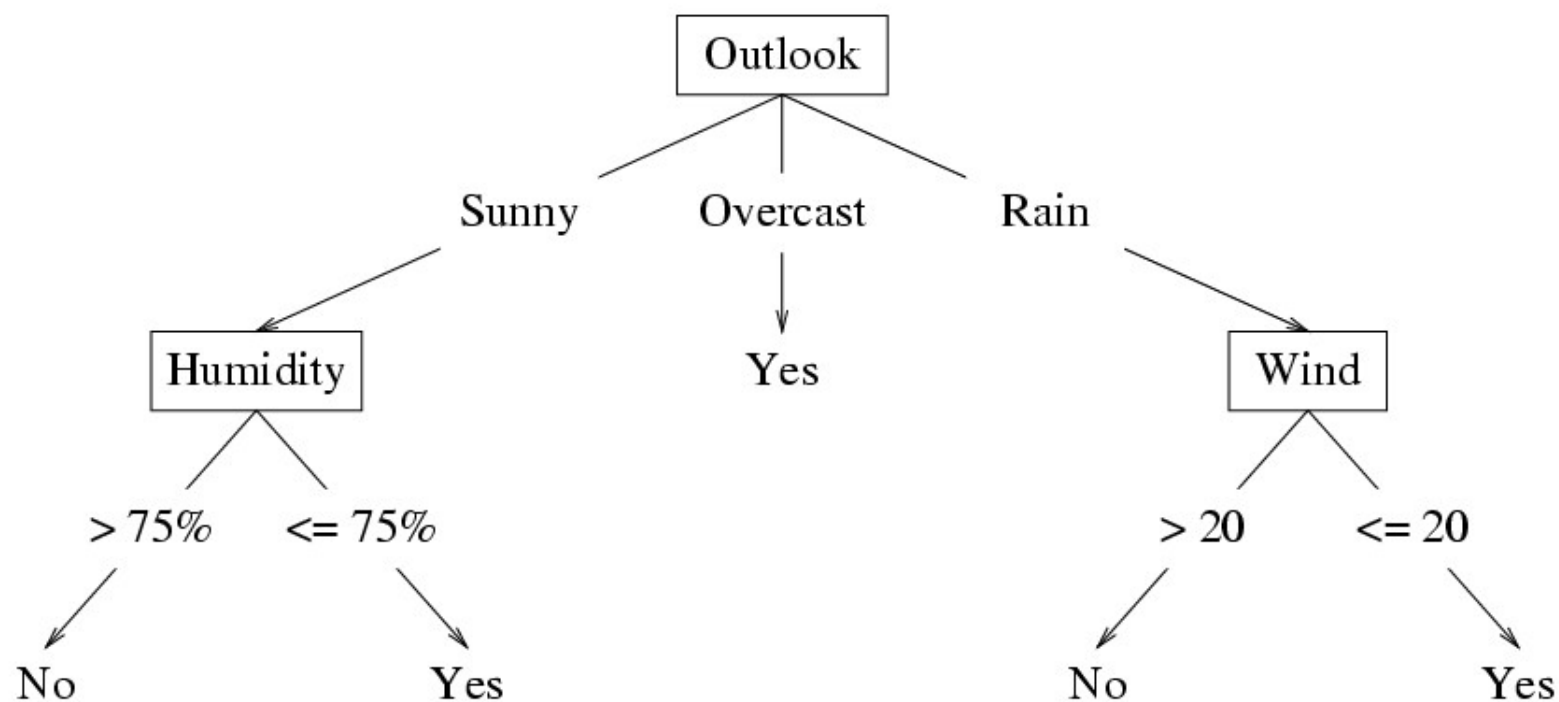
Decision Tree Hypothesis Space

- **Internal nodes** test the value of particular features x_j and branch according to the results of the test.
- **Leaf nodes** specify the class $h(\mathbf{x})$.



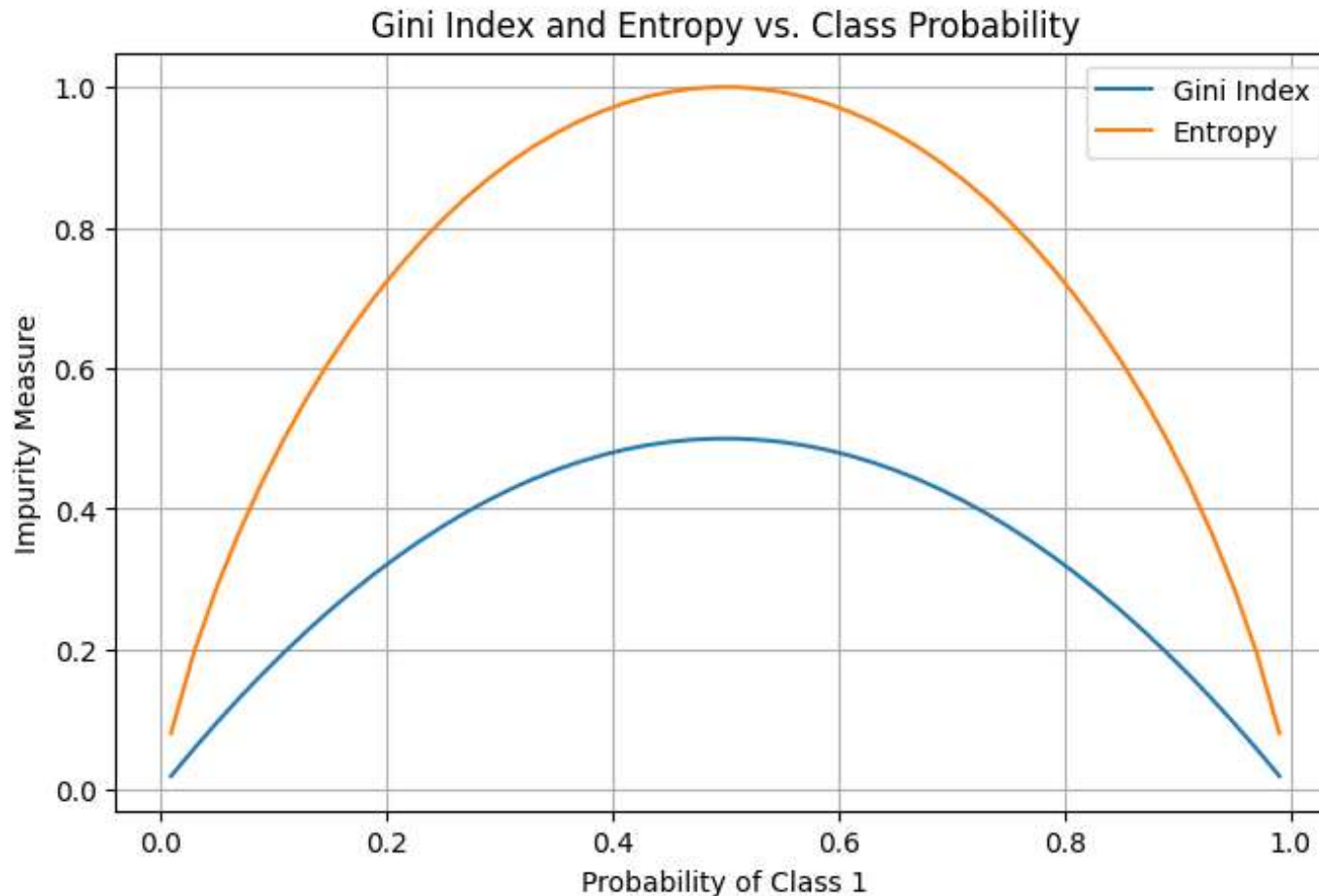
Suppose the features are **Outlook** (x_1), **Temperature** (x_2), **Humidity** (x_3), and **Wind** (x_4). Then the feature vector $\mathbf{x} = (\text{Sunny}, \text{Hot}, \text{High}, \text{Strong})$ will be classified as **No**. The **Temperature** feature is irrelevant.

If the features are continuous, internal nodes may test the value of a feature against a threshold.



1. If all the examples are in the same class, we will return the class label.
2. If there are no features left, we have noisy data. We can either return the majority decision or make a decision probabilistically.
3. If there are no examples left, then some combination of input features is absent in the training set. We can use the examples at the parent node to make a decision. If the examples are not in the same class, we can either return the majority decision or make a decision probabilistically.

Gini index



Gini index:

- computationally faster
- favors splits with result in balanced distributions

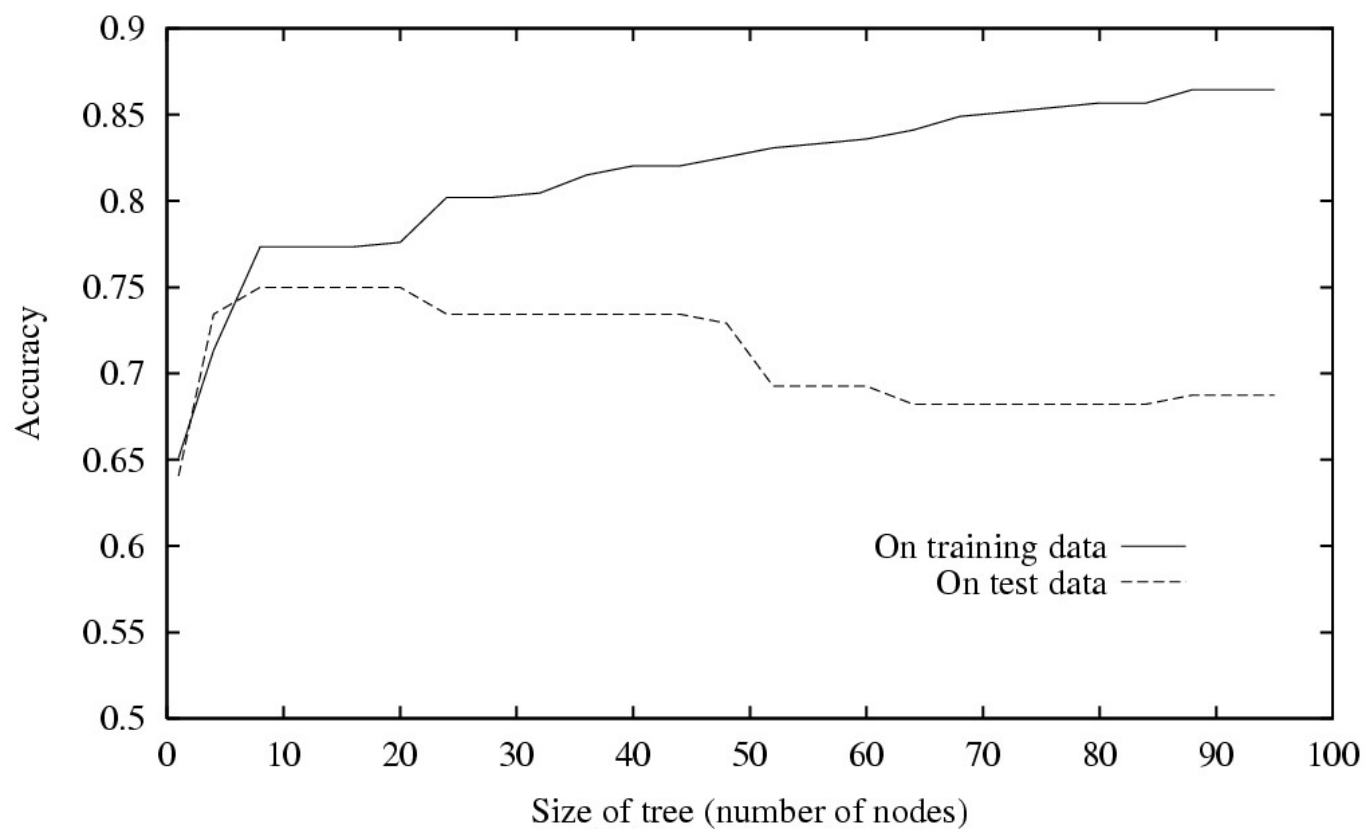
Gini Index

$$I_G = 1 - \sum_{j=1}^c p_j^2$$

Entropy

$$I_H = - \sum_{j=1}^c p_j \log_2(p_j)$$

Overfitting in Decision Trees



Overfitting

- A decision tree keeps splitting the data to minimize error. If allowed to grow without restrictions, this makes the model fit the training data
- However, it performs poorly on new/unseen data, because it has learned **noise** instead of general patterns.

How to Prevent Overfitting

- **Limit Tree Depth**
 - Use `max_depth` to restrict how deep the tree can grow.
- **Minimum Samples per Split / Leaf**
- **Pruning**
 - Remove branches that have little impact on prediction accuracy (post-pruning).

Pre-pruning (Early Stopping)

- Sets limits (stopping criteria) like **maximum depth**, minimum samples per leaf, or minimum information gain, halting growth when met.
- During the tree-building process
- Pros: More computationally efficient as it avoids building a large tree.
- Cons: Can stop too early (horizon effect), potentially missing important splits, leading to underfitting.

Post-pruning

- Removes nodes/subtrees that don't significantly improve performance on a validation set, often using methods like cost-complexity pruning.
- After the tree is fully grown.
- Pros: Often finds a better balance, resulting in a more accurate model that generalizes well.
- Cons: More computationally intensive as it first builds a complex tree

Decision Tree

Advantages

- Easy to understand and **interpret**.
- No need for data normalization or scaling.
- Works for both numeric and categorical data.

Disadvantages

- Can **overfit** easily (fit too closely to training data).
- Small changes in data can cause large changes in the tree.
- Often outperformed by **ensemble** methods like **Random Forests**.

ID3

- Splitting Criterion: **Information Gain** (based on **Entropy**).
- Data Types: Works only with categorical (discrete) attributes and target variables; continuous data must be discretized first.
- Tree Structure: Can create multi-way splits (more than two branches per node).
- Limitations: Prone to **overfitting**, doesn't handle missing values well, and favors attributes with many values.

C4.5 (Classification 4.5)

- Splitting Criterion: **Gain Ratio** (to overcome ID3's bias towards attributes with many values).
- Data Types: Handles both categorical and continuous attributes (by creating intervals).
- Improvements: Includes pruning (to reduce overfitting) and handles missing values.
- Successor: C5.0 is a faster, more efficient version.

CART (Classification and Regression Trees)

- Splitting Criterion: **Gini Impurity** (or cross-entropy).
- Data Types: Handles both categorical and continuous features.
- Tree Structure: Builds **binary** trees (only two branches per node).
- Tasks: Versatile, used for both Classification and Regression (predicting continuous values).

Feature	ID3	CART
Split criterion	Entropy / Information Gain	Gini / MSE
Data types	Categorical only	Categorical + Numerical
Tree structure	Multi-branch	Binary
Tasks	Classification only	Classification + Regression
Pruning	No	Yes